

Monte Carlo — auditoría de rendimiento y memoria

AlgoProject · módulo `strategies/monteCarlo`

Dónde se le va el tiempo, dónde se le va la memoria, y qué cambiar.

Medido sobre `XAUUSD / MC Trades` — 757 estrategias, 960.705 operaciones.

Generado el 2026-09-20. Ninguna línea del módulo fue modificada.

Qué es este documento. El módulo `strategies/monteCarlo` acaba de ser reorganizado y va a tener que procesar miles de estrategias con cientos de miles de simulaciones cada una. Este informe dice **dónde se le va el tiempo y dónde se le va la memoria**, con los números medidos que lo demuestran, y propone qué cambiar y en qué orden.

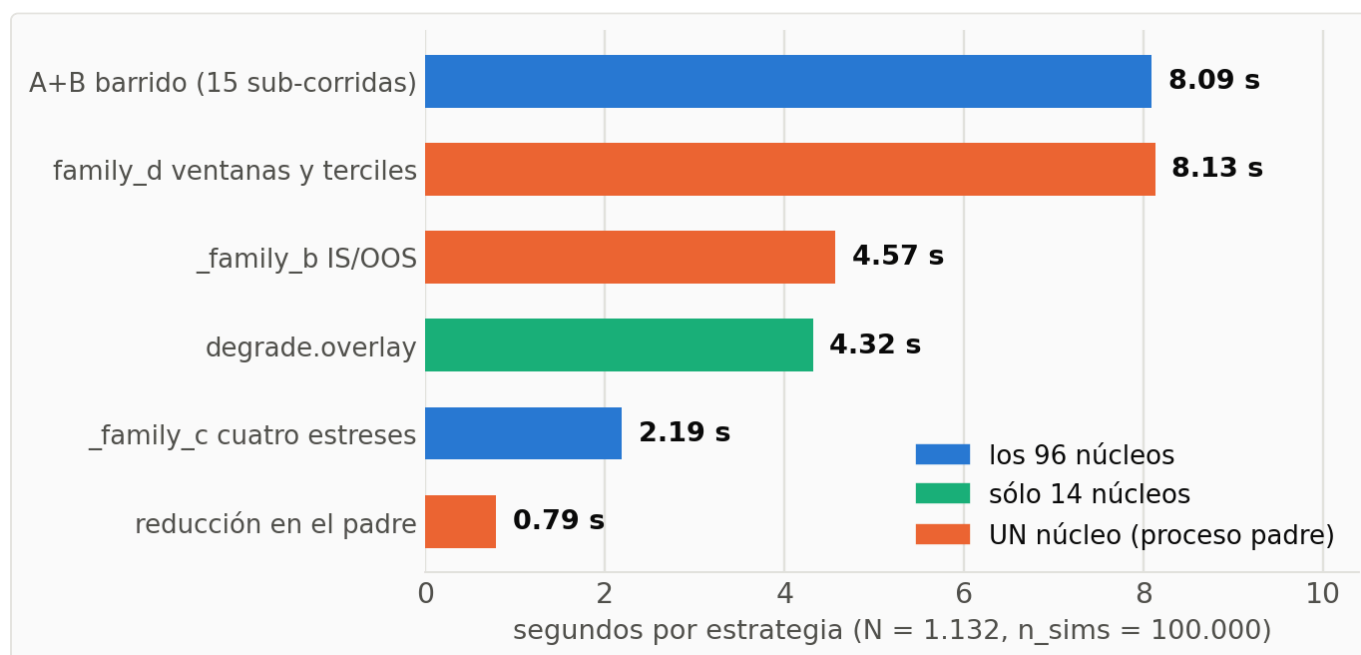
No se ha tocado ni una línea del módulo. Todo lo que aparece aquí como "propuesta" está prototipado y medido fuera del repositorio, en el scratchpad de la sesión. Lo que aparece como "medido" viene de ejecutar el código tal y como está hoy.

Sobre qué se midió. Databank `MC Trades` del proyecto `XAUUSD` — 757 estrategias, 960.705 operaciones, exportadas el 2026-09-19 a `~/Desktop/AlgoData/raw/XAUUSD/MC_Trades/2026-09-19/`. Distribución de operaciones por estrategia: mínimo 542, primer cuartil 882, mediana 1.132, tercer cuartil 1.610, percentil 95 2.374, máximo 3.437. Configuración: la de `strategies/monteCarlo/config.yaml` vigente, con `n_sims: 100000`, `chunk: 2000`, `max_workers: null`.

Generado el 2026-09-20.

1. El punto de partida

Una estrategia mediana (N = 1.132 operaciones) tarda **28,2 segundos**. Multiplicado por las 757 del databank, y corrigiendo por la operación media real (1.269 frente a 1.132), el run completo sale a **unas 6,2 horas**.



Cómo leer la figura. Cada barra es una parte del análisis de una estrategia. El color dice cuántos núcleos de los 96 usa esa parte mientras corre: azul, todos; verde, sólo catorce; naranja, **uno solo**. Las tres barras naranjas suman 13,5 de los 28,2 segundos. Casi la mitad del tiempo, esta máquina de 96 núcleos está usando uno.

El renderizado no aparece porque no se ve: la página HTML de una estrategia cuesta 24 milisegundos y `scoring.verdict()` menos de uno. El problema es todo simulación.

El presupuesto de caminos simulados

Una estrategia consume 4.362.000 caminos simulados. Repartidos así:

parte	caminos	%	dónde corre
Barrido A+B (15 sub-corridas × 100.000)	1.500.000	34,4 %	pool
<code>degrade.overlay</code> (14 × 100.000)	1.400.000	32,1 %	pool, sólo 14 tareas
Family D ventanas y terciles (36 × 20.000)	720.000	16,5 %	padre, 1 núcleo
Family C cuatro estreses (4 × 100.000)	400.000	9,2 %	pool
Family B IS/OOS (2 × 100.000)	200.000	4,6 %	padre, 1 núcleo
<code>stitch</code> camino adversarial (7 × 20.000)	140.000	3,2 %	padre, 1 núcleo
<code>fan</code> cono de equity	2.000	0,0 %	padre

Aparte, `stability.spread()` gasta 4,8 millones de caminos más, pero **una sola vez por databank**, no por estrategia. Cuesta 26 segundos y usa 8 de los 96 núcleos.

2. Por qué el servidor no rinde: no es SQX, es la memoria

La primera hipótesis razonable es que StrategyQuant X, abierto en la GUI, esté acaparando la máquina. **En CPU no lo hace. En memoria sí, y mucho.**

CPU: nadie reserva nada

- Afinidad `0-95` para el shell, para Python y para el propio SQX. Sin `cpu.max`, sin `cpuset`, sin `memory.max` en el cgroup.
- Con la GUI abierta y ociosa, SQX consume **0,8 % de un núcleo**.
- Durante la prueba saturada, **85,6 de los 96 núcleos figuran ocupados**. Los núcleos están ahí y Python los tiene.

Un aviso que sí importa: SQX arranca con **95 hilos comput aparcados** (`Preparing thread executors: 95` en su propio log). Ociosos no molestan, pero un build en la GUI pide la máquina entera igual que el módulo. **Los dos no caben a la vez.**

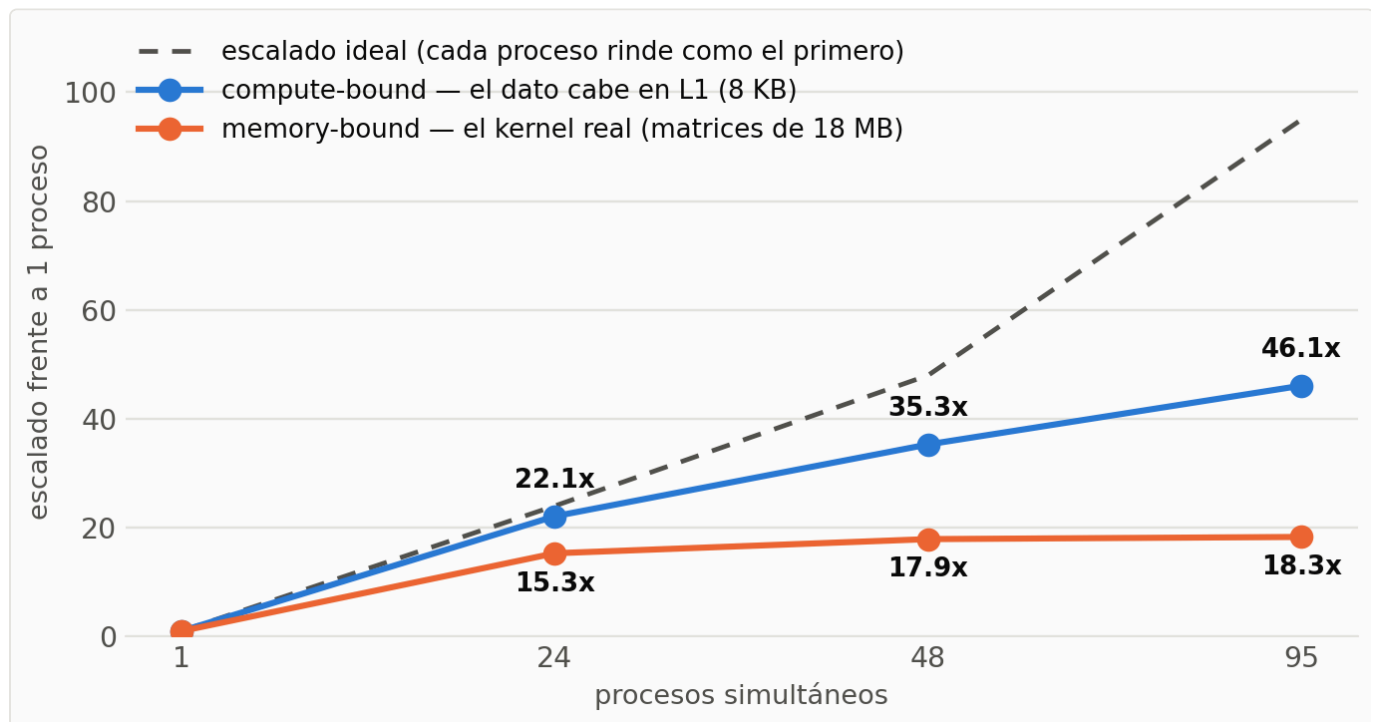
96 núcleos son 48

AMD EPYC 7413: 2 sockets × 24 núcleos × 2 hilos SMT = 96 lógicos, **48 físicos**. Los hermanos SMT son `cpu N ↔ cpu N+48`. Un solo nodo NUMA. Y el segundo hilo de cada núcleo no aporta nada en esta carga — resta:

reparto	caminos/s
96 procesos sobre los 48 núcleos físicos	211.419
96 procesos sobre los 96 lógicos (físicos + SMT)	203.617

Los núcleos están libres; lo que falta es ancho de banda

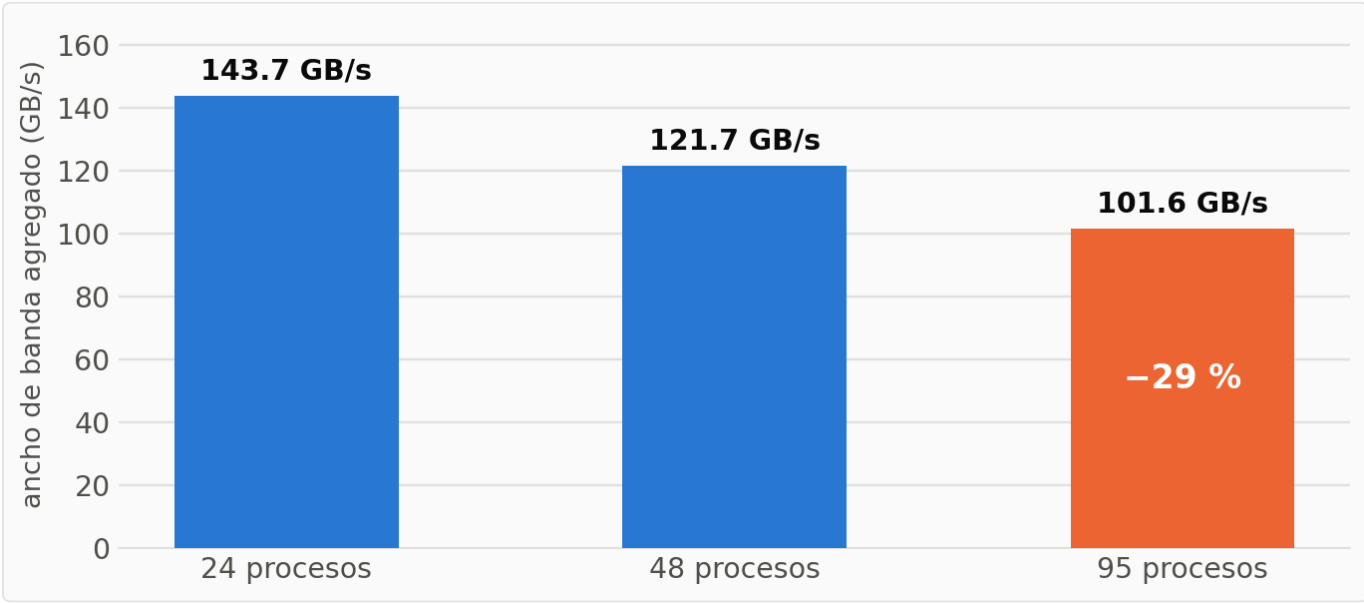
Ésta es la medición que lo zanja. El mismo pool, el mismo número de procesos, la misma operación — sólo cambia si el dato cabe en caché o hay que ir a buscarlo a la DRAM:



Cómo leer la figura. El eje vertical es cuántas veces más trabajo hace el conjunto frente a un solo proceso. La línea discontinua es lo que pasaría si cada proceso rindiera como el primero. La azul es un cálculo que trabaja sobre 8 KB, que caben en la caché L1 de cada núcleo: escala 46,1×. La naranja es el kernel real del módulo, con matrices de 18 MB que no caben en ninguna caché: **escala 18,3× y se queda plano desde los 24 procesos.**

La diferencia entre las dos líneas no es CPU. Los 85,6 núcleos ocupados que registra el sistema están ocupados **esperando memoria.**

Y pedir más procesos empeora el ancho de banda



Cómo leer la figura. Es una triada tipo STREAM — la prueba estándar de ancho de banda de memoria: leer dos vectores, multiplicar uno por un escalar, sumar y escribir el resultado. Mide los GB por segundo que el conjunto de procesos consigue mover de verdad. El máximo está en 24 procesos. **Con 95 procesos la máquina mueve un 29 % menos de datos que con 24**, por contención en los controladores de memoria.

`max_workers: null` resuelve a `os.cpu_count()` = 96, que es exactamente el peor punto de esa curva.

Memoria: aquí SQX sí reserva, y es enorme

```
~/Desktop/SQX/StrategyQuantX.config: option -Xmx108g
~/Desktop/SQX/sqcli.config:          option -Xmx32g
user/settings/settings.xml:          <memoryCleanup>false</memoryCleanup>
```

La GUI tiene permiso para crecer hasta **108 de los 125 GB**. Ahora mismo ocupa **71,5 GB, todos anónimos** — heap de la JVM, sólo 0,1 GB respaldado por fichero. Eso significa que el kernel **no puede recuperar ni un byte** mientras la GUI esté abierta: no es caché, es memoria comprometida. Y `memoryCleanup=false` le dice que ni lo intente.

RAM total	125,4 GB
SQX GUI (heap anónimo, irrecuperable)	-71,5 GB
Pools huérfanos del módulo (ver §5)	-8,9 GB
VS Code, Xorg, sesiones de agente	--5 GB
disponible para el Monte Carlo	37,4 GB (y 1,2 GB de swap libre)

Y el worker de SQX pide otros 32 GB cuando arranca. Si coincide con el módulo: 71,5 + 32 + 32 = 135 GB sobre 125. No cabe.

3. Dónde se va la memoria del módulo: toda en los trabajadores

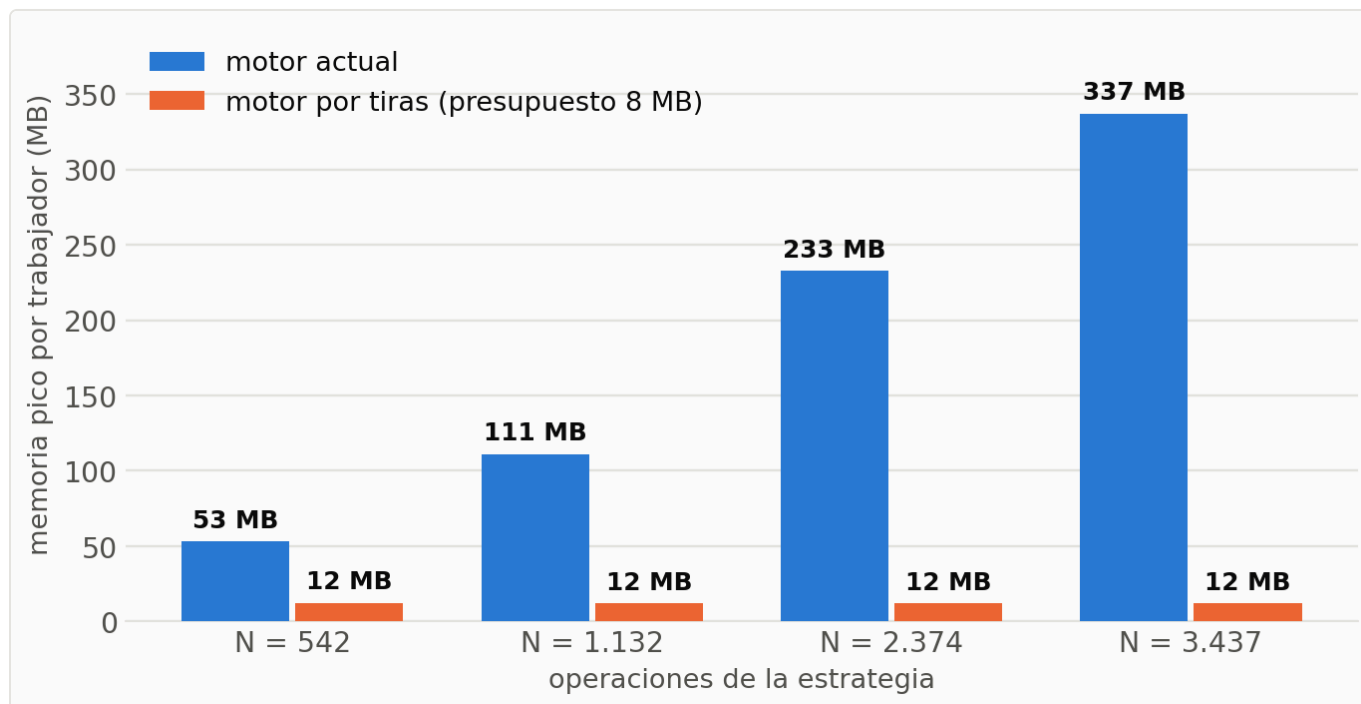
El proceso padre **no es el problema**, y conviene decirlo porque es donde uno mira primero:

	N = 1.132	N = 3.437
Arrays crudos en <code>sweeps.execute</code>	96,0 MB	96,0 MB
Resultado que devuelve <code>analyse()</code>	1,4 MB	1,5 MB
JSON de caché del explorador	0,7 MB	0,8 MB
RSS máximo del padre	0,74 GB	1,52 GB

Lo que el padre guarda no depende de N: son 15 etiquetas × 8 estadísticos × 100.000 simulaciones, 96 MB fijos.

El problema es `_batch`, que vive con **6,1 matrices (chunk, N) de float64 a la vez**. Medido con `tracemalloc`, en unidades de "matriz base" (`chunk × N × 8 bytes`):

asignación	matrices	por qué
<code>draws.iid_bootstrap</code>	1,0	la matriz de índices y nada más
<code>draws.stationary</code>	2,1	<code>fresh</code> (float64) + <code>starts</code> (int64) + <code>out</code> (int64)
<code>draws.block_shuffle</code>	3,2	el <code>argsort</code> estable que quita el relleno
<code>draws.block_bootstrap</code>	2,2	
<code>stress.skip</code>	1,1	
<code>stress.cost_shock</code> / <code>spread_widen</code>	3,0	
<code>stress.fill_degrade</code>	3,1	
<code>metrics.paths</code>	5,1	<code>equity</code> , <code>peak</code> , <code>drop</code> , <code>drop/peak</code> , y dos <code>np.where</code> para <code>wins/loss</code> — encima de su entrada
<code>_batch</code> completo	6,1	



Cómo leer la figura. Azul, lo que pide hoy un trabajador para procesar un lote de 2.000 simulaciones, según cuántas operaciones tenga la estrategia. Naranja, lo que pediría con el motor propuesto. La barra azul crece con N porque el lote se mide en simulaciones; la naranja no crece porque se mide en bytes.

Con `chunk: 2000` y 96 trabajadores eso son **10,7 GB** en la estrategia mediana y **32,3 GB en la más larga**, contra 37,4 disponibles. El run no se cae al empezar: se cae a mitad de las 757, cuando le toca una estrategia larga.

`chunk` está documentado en el código como "una decisión de memoria, no estadística". Acota simulaciones, no bytes — y por eso el pico depende de qué estrategia toque.

4. La propuesta: procesar por tiras que quepan en caché

Bajar `chunk` es lo obvio y es lo equivocado: reduce la memoria pero multiplica las tareas y el tráfico de memoria sigue igual. Lo que propongo es partir el lote **dentro del trabajador** en tiras de filas, con buffers preasignados que se reutilizan, en `float32 / int32`, y dimensionar la tira por un **presupuesto en bytes**.

Tres consecuencias, las tres medidas sobre un prototipo funcionando:

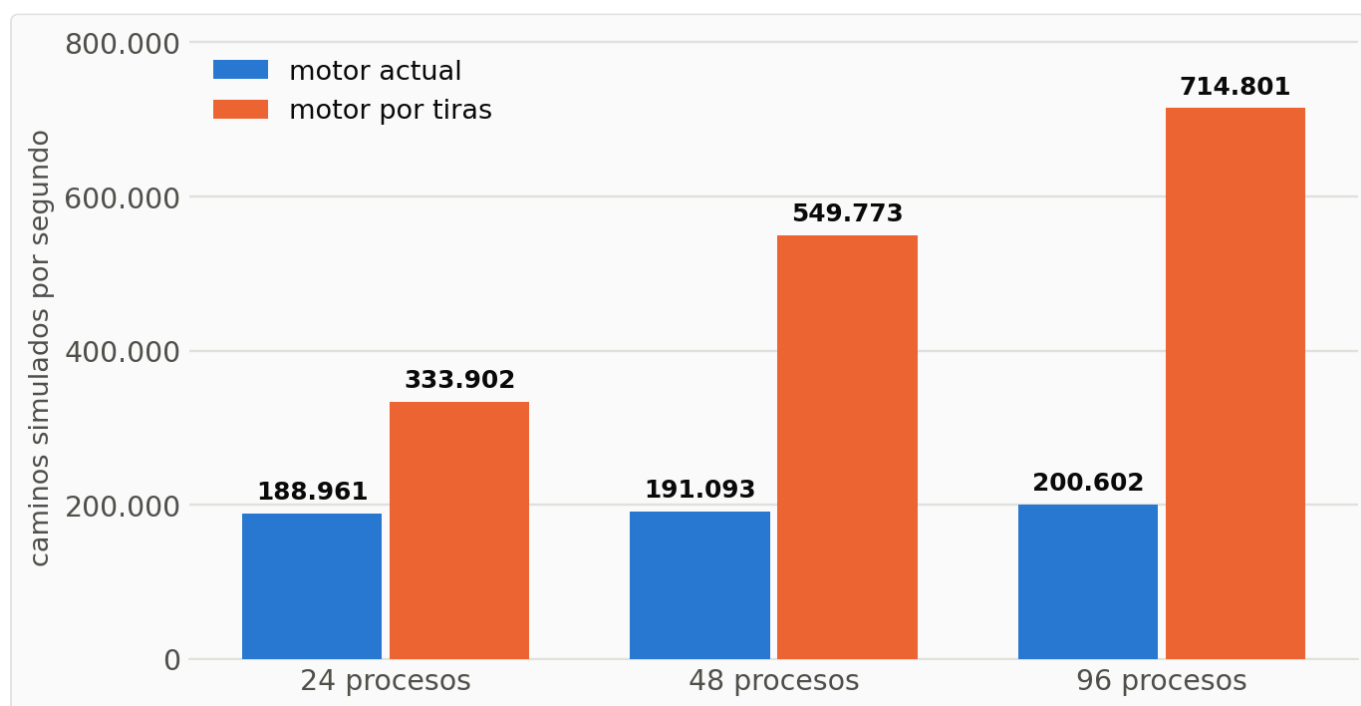
4.1 La memoria se vuelve constante

N	hoy	por tiras	ratio
542	53,2 MB	12,2 MB	4,4×
1.132	111,1 MB	12,1 MB	9,1×
2.374	232,8 MB	12,1 MB	19,2×
3.437	336,9 MB	12,0 MB	28,0×

Deja de importar si la estrategia tiene 500 operaciones o 3.500.

4.2 El mismo cambio rompe el techo de ancho de banda

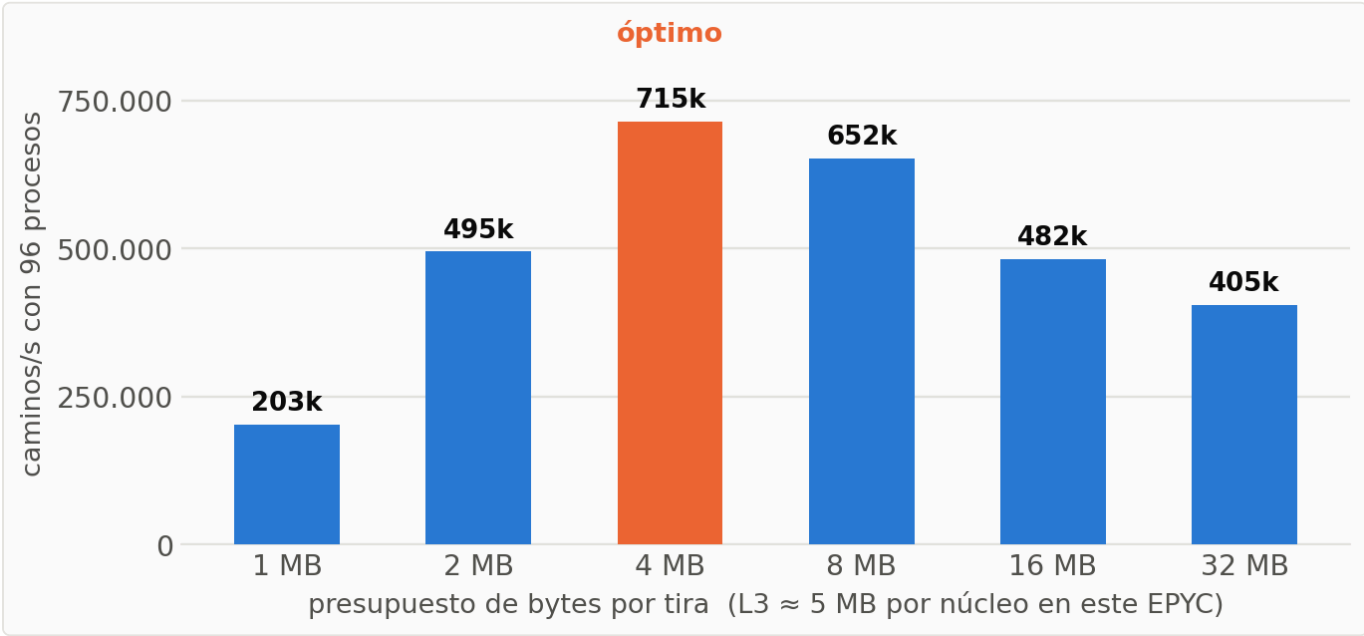
Al caber el conjunto de trabajo en L3, el kernel deja de ir a DRAM:



Cómo leer la figura. Caminos simulados por segundo, sumando todos los procesos. El motor actual (azul) está plano: 24, 48 o 96 procesos dan lo mismo, porque el límite no son los núcleos. El motor por tiras (naranja) **sigue escalando hasta los 96**. De 200.602 a 714.801 caminos/s: **3,6× con el mismo hardware**.

Esto **invierte** la recomendación de bajar `max_workers` a 24-48. Era correcta para el kernel actual, limitado por DRAM. Deja de serlo en cuanto el kernel cabe en caché: con tiras, los 96 sí valen.

4.3 El tamaño de la tira tiene un óptimo claro



Cómo leer la figura. Cada barra es un presupuesto de bytes distinto para la tira, midiendo con la máquina saturada a 96 procesos. La L3 de este EPYC son unos 5 MB por núcleo. Por debajo de 4 MB manda el intérprete de Python: tiras de 36 filas significan 56 iteraciones por lote y el coste por iteración se come la ganancia. Por encima de 8 MB la tira se sale de L3 y vuelve el problema de siempre. **El óptimo es 4 MB.**

4.4 Presupuesto de memoria resultante

	hoy (peor caso)	propuesto
Trabajadores	96 × 337 MB = 32,3 GB	96 × 6 MB = 0,6 GB
Padre	1,52 GB	1,52 GB (0,3 con <code>n_sims</code> a 20.000)
Total	~33,8 GB — no cabe en 37,4 con margen	~2,1 GB

5. Dos avisos de corrección, verificados

`float32` no degrada nada que decida — y arregla el invariante

Los percentiles que alimentan los vetos coinciden con `float64` con error relativo entre $4 \cdot 10^{-8}$ y $5 \cdot 10^{-7}$:

número que decide	float64	float32	error relativo
<code>dd_pct</code> percentil 95	0,135687	0,135687	$5,45 \cdot 10^{-7}$
<code>dd_pct</code> percentil 99	0,178404	0,178404	$3,08 \cdot 10^{-7}$
<code>net</code> percentil 5	8.104,798000	8.104,798828	$1,02 \cdot 10^{-7}$
<code>pf</code> percentil 5	1,050673	1,050673	$4,15 \cdot 10^{-8}$

Eso son cinco órdenes de magnitud por debajo del 3 % que los mismos números ya se mueven entre corridas independientes, porque el módulo no lleva semilla a propósito.

Y hay una mejora inesperada. `sweeps.invariant()` exige que un modelo que sólo reordena deje el beneficio neto idéntico, y el README dice "cero salvo coma flotante". Medido sobre 20.000 reordenaciones de una estrategia real:

representación	desviación típica del neto
<code>float64</code> (hoy)	$7,52 \cdot 10^{-12}$ USD
<code>float32</code> sumado en <code>float64</code>	0,0 exacto
<code>float32</code> sumado en <code>float32</code>	$7,70 \cdot 10^{-3}$ USD

Con 24 bits de mantisa sumados en `float64` no hay redondeo en ningún momento, así que el invariante pasa a ser exacto en vez de aproximado.

El acumulador tiene que ser `float64` explícito — `sum(..., dtype=np.float64)`, `einsum(..., dtype=np.float64)`. Con acumulación en `float32` esa desviación sube a $7,7 \cdot 10^{-3}$ USD y el invariante deja de serlo, que es justamente el número que el README declara que sólo puede ser cero.

El sorteo por tiras no puede reciclar uniformes

Al escribir el prototipo aparece la tentación de usar el mismo buffer de números aleatorios para "quién reinicia el bloque" y "dónde reinicia". **Sesga:** las posiciones de arranque quedan correlacionadas con el umbral `1/block` y concentradas al principio de la serie. Hacen falta dos buffers independientes.

Con dos, el sorteo es estadísticamente idéntico al actual: fracción de operaciones consecutivas 0,9093 en el prototipo contra 0,9089 en la referencia, con un valor teórico de 0,9091 para un bloque medio de 11.

Un pool que nunca se cierra deja procesos vivos para siempre

`simulate/engine.py` guarda el pool en `_POOL` y no lo cierra nunca — no hay `shutdown()` ni `atexit` en todo el repositorio. Si el padre muere sin desenrollar la pila (Ctrl-C duro, `kill -9`, el panel Flask cerrado desde la terminal), el `forkserver` y sus trabajadores quedan **reparentados a `systemd` y vivos indefinidamente**.

En esta máquina había **68 procesos huérfanos ocupando 8,9 GB**. Uno de ellos llevaba nueve días y tenía la ruta de módulo **anterior a la reorganización** (`strategies.monteCarlo.engine`), lo que prueba que sobreviven a cualquier cosa.

```
ps -eo pid,ppid,etime,rss,cmd | grep -E 'forkserver|resource_tracker' | grep -v grep
```

Se limpian matando por PID el padre `forkserver` cuyo PPID sea 1. Nunca `pkill -f python3`: el patrón se alcanza a sí mismo.

6. `n_sims: 100000` está cinco veces por encima de lo que el módulo exige

El módulo ya lleva su propio juez: `stability.spread()` recalcula los percentiles que disparan los vetos ocho veces de forma independiente y mide cuánto se mueven. Su tolerancia declarada en `config.yaml` es el 10 %. Medido sobre la estrategia mediana:

<code>n_sims</code>	tiempo	dispersión relativa máxima	¿cumple el 10 %?
100.000	26,1 s	2,99 %	sí
50.000	13,2 s	5,00 %	sí
20.000	5,0 s	8,32 %	sí — el primero que lo cumple justo
10.000	2,4 s	11,78 %	no
5.000	1,3 s	11,00 %	no

Y hay más margen del que parece, porque no todas las corridas alimentan un veto:

- `degrade.overlay` **gasta 1,4 millones de caminos (32 % del presupuesto) sólo para dibujar histogramas.** `scope_shape()` devuelve únicamente lo que produce `metrics.shape()`: 60 cuentas de bin, la mediana y un percentil. Un histograma de 60 bins está estable a 5.000 caminos.
- **El barrido de 12 tamaños de bloque** **gasta 1,2 millones más para dibujar una curva de sensibilidad.** Ni `gates.py` ni `scoring.py` lo leen: los vetos sólo miran `stationary` (la corrida titular) e `iid_bootstrap` (la línea base de composición).

Propuesta: tres presupuestos declarados en `config.yaml` en vez de uno.

clave	valor	para qué
<code>n_sims</code>	20.000	lo que decide: corrida titular, línea base, los cuatro estreses, IS/OOS
<code>n_curve</code>	10.000	el barrido de tamaños de bloque — una curva, no una cola
<code>n_shape</code>	5.000	los histogramas de <code>degrade.overlay</code>

El presupuesto por estrategia cae de 4.362.000 a unos 1.290.000 caminos.

Efecto de segundo orden que conviene anticipar. En cuanto se aplique esto, Family D pasa a ser el 67 % del presupuesto restante — y es justo la parte que hoy corre en un solo núcleo. A partir de ahí, sacarla del proceso padre deja de ser opcional.

7. Lo que NO es un cuello de botella

Comprobado, para que nadie lo vuelva a mirar:

sospechoso	medida real	veredicto
El renderizado de la página HTML	24 ms por estrategia, 0,65 MB de HTML	irrelevante
El <code>pickle</code> que <code>degrade.overlay</code> manda al pool	2,2 MB en total, 0,17 ms por envío	irrelevante
Leer los 757 CSV por adelantado en <code>report.py</code>	4,5 s y 0,17 GB	aceptable
<code>scoring.verdict()</code>	menos de 1 ms	irrelevante
El arranque en frío del pool con <code>forkserver</code>	0,82 s, una vez	correcto y bien resuelto
La memoria del proceso padre	0,74–1,52 GB	no es el problema

El `forkserver` con preload de `strategies.monteCarlo.simulate.engine` está bien planteado y funciona: los trabajadores arrancan con numpy ya importado.

Un detalle latente, no activo: OpenBLAS está compilado con `MAX_THREADS=64`. Hoy no molesta porque el kernel no hace llamadas BLAS, pero con `vol_model: garch` cada trabajador podría abrir 64 hilos. `OMP_NUM_THREADS=1` en el entorno de los trabajadores es un seguro barato.

8. Qué hacer, en orden

- `_batch` por tiras con buffers persistentes.** Es el cambio que da las dos cosas a la vez: 28× menos memoria en el peor N y 3,6× de rendimiento. Todo lo demás del módulo sigue igual — mismas firmas, mismos arrays de salida.
- `chunk` deja de ser el knob de memoria** y pasa a ser sólo granularidad de tarea. El knob nuevo es `tile_bytes: 4_000_000`, y lo que declare se cumple sea cual sea N.
- Tres presupuestos de simulación** (`n_sims`, `n_curve`, `n_shape`), justificados por la propia medida de estabilidad del módulo, no por opinión.
- Sacar del proceso padre el trabajo monohilo** — `family_d`, `_family_b`, `stitch`. En un batch de 757 estrategias el paralelismo natural es *una estrategia por proceso*, no un sub-test cada vez: elimina de golpe las 19 barreras de sincronización por estrategia y los tres caminos de un núcleo. El panel interactivo se queda como está, porque ahí sí interesa paralelizar dentro de una estrategia.
- `atexit` sobre el pool** y un `try/finally` en `jobs._run`, para que no vuelvan a quedar huérfanos.
- `max_workers: null (96)` se queda** — pero sólo después de (1). Antes de (1) es el peor punto de la curva de ancho de banda.
- Decidir qué hacer con `-Xmx108g`**. Es decisión del dueño de la máquina, no del módulo. Pero 108 GB reservados para una GUI que usa 71,5 y nunca devuelve dejan al análisis con un tercio del servidor. Para correr las 757 estrategias: o la GUI se cierra, o ese número baja.

Estimación

escenario	caminos/estrategia	757 estrategias
Hoy	4.362.000	6,2 h
Techo de la máquina con el presupuesto actual	4.362.000	3,4 h — el orquestador desperdicia el 45 %
+ (4) paralelizar por estrategia	4.362.000	~3,4 h
+ (1) motor por tiras	4.362.000	~0,9 h
+ (3) tres presupuestos de simulación	~1.290.000	~20 min

Apéndice — dónde está cada cosa

Los hallazgos de este informe están además escritos en `knowhow/07-practices.md` , con sus etiquetas de procedencia (verificado por prueba directa), bajo tres secciones:

- *El kernel Monte Carlo por tiras: 84× menos memoria y 3,6× más rápido, a la vez*
- *Este servidor tiene 48 núcleos, no 96, y SQX se queda con 71 GB de los 128*
- *Un `ProcessPoolExecutor` global sin apagado deja el pool vivo cuando el padre muere*

El export que sirvió de banco de pruebas está anotado en `~/Desktop/AlgoData/INDEX.md` .

Los ficheros del módulo que cambiarían si se aceptan las propuestas son tres: `simulate/engine.py` (las tiras y el pool), `model/draws.py` (los sorteos) y `simulate/metrics.py` (el kernel de estadísticos). `config.yaml` gana `tile_bytes` , `n_curve` y `n_shape` . Ningún fichero de `verdict/` ni de `render/` se toca.